

6/15 – Professional Services Intro, Partnerships & Argos Workshop Overview

Professional Services Intro

Team Introductions

- Ronald Franklin – PS engineer, over 10 years at Evisions, previously a programmer analyst at a university in Tennessee for 10 years
- Jonathan Kim – PS engineer, over 10 years at Evisions, works closely with sales, implemented the Ellucian Banner system at a university prior to Evisions
- Gail – PS trainer, approximately 11.5 years at Evisions, worked at a university for 15 years prior, has trained hundreds of clients
- Most of the PS team has a higher ed background, which is a major asset since the client base is overwhelmingly higher ed institutions

Today's Schedule

- Hour 1 – PS team introduction and overview (Ronald, Jonathan, Gail)
- Hour 2 – Partnerships session with James Hawking
- Afternoon – Argos training begins with Gail, continuing Tuesday through Thursday
- Thursday afternoon – wrap up meeting with Ronald, Jonathan, Gail, and the rest of the group
- Friday and the following Monday are a company holiday

What Is Professional Services (PS)

- **Definition** – services provided by individuals or teams with specialized expertise in an industry, delivered in a consulting, advisory, or specialized nature
- In short, PS can do the client's job for them when the client lacks the time or expertise
- Delivered remotely and on site, depending on client needs
- Billed hourly, by project, and by specific service agreements
- **Examples of professional services more broadly** – lawyers, CPAs, tax consultants, cybersecurity consultants; common thread is billing by hour for specialized expertise
- Evisions PS specifically provides software development support and security consulting

Evisions PS Team Overview

- Team of approximately 10–11 people
- Most have a higher ed background, since that is the overwhelming client base
- Team combines three strengths:
 - Higher ed experience
 - Product expertise (especially Argos)
 - Technical skills
- Clients often lack one or more of these, so PS fills the gap (eg new hires who need to be taught their own job functions)
- Client facing team that contributes to Evisions revenue through billable services

Core PS Service Offerings

- **Training** – teaching clients how to use the tool itself, not building their end product for them; remote training is common, done over Zoom
- **Consulting** – advising clients on best practices for their job, often involves more planning and executive-level discussions

- **Development** – building something for the client, most often a data block or report
- **Implementation** – configuring and installing a purchased product (most often Argos, but also IntelliCheck and FormFusion) and teaching the client how to roll it out on campus
- Services often overlap on a single project (eg a new Argos purchase typically includes implementation, training, and ongoing consulting/development), though some contracts are for one service only
- Training analogy – learning to use a tool (eg Word) is separate from using that tool to write a specific document; PS teaches the tool, not the end deliverable

Why PS Matters to Clients

- Helps clients through complex change, such as implementing a large product like Banner
- Higher ed IT departments are shrinking and increasingly outsourcing technical work to consulting firms
- PS enables faster staff training and implementation, maximizing the value of a technology purchase
- Example risk – a school purchased Argos but let it sit unused because they planned to self-implement and never did
- PS tailors services to meet institutional goals rather than offering a one-size-fits-all package
- Goal is client success and renewal – happy, successful clients renew licenses and return for more training and services

Team Structure – Roles

- **Trainers** – provide instruction for fast adoption of the software; help clients move beyond their comfort zone to use more of the tool’s capabilities
- **Engineers** – provide development services, such as building a finance or budget report; most work is done remotely with frequent client check-ins via Zoom
- **Consultants** – build strategic partnerships with clients, more talking and planning involved, on site and remote work
- Pre-2020 vs post-2020 – training was almost entirely in person before 2020 (Monday travel, Tuesday through Thursday on site, Friday return); now mostly remote, even when a client requests on-site delivery

Skills Needed by Role

- **Trainers** – strong public speaking skills, deep tool knowledge to handle live questions, understanding of the client’s business context, some technical skills, strong organizational skills for building training materials
- **Engineers** – technical focus, client-facing and one-to-one communication skills to gather requirements, computer science or engineering background preferred, core technical skill is SQL (Structured Query Language), most clients use Oracle databases but other database types are also supported
- **Consultants** – similar technical foundation to engineers, plus extensive planning and executive-level discussion experience; typically mid-career professionals with 25–30 years of experience, considered experts in their field and the highest paid of the three roles
- Team members are ”jack of all trades” – Gail, Jonathan, and Ronald have all performed training, consulting, and development at various points

Client Experience and Handoff Flow

- Sales process → PS delivery → customer success
- **Sales** – works with the client to determine needs (new product purchase or additional service

for an existing product); Jonathan works closely with sales

- **PS** – delivers the actual services (training, development, consulting, implementation)
- **Customer Success** – ensures ongoing engagement and full product adoption after the PS project; retaining clients is prioritized over acquiring new ones

Project Management

- Projects are managed in collaboration with clients, sales, and finance
- All work is tracked via project time entries; billing is how PS sustains itself and generates company revenue
- Workflow: sales hands off project to PS in Salesforce → kickoff call with client to discuss priorities and dates → remote access to client systems as needed → ongoing development work with tracked time, notes, and communication
- Projects are scoped with an hour estimate (eg 20 hours for a report); PS cannot exceed that estimate unless requirements change and additional hours are negotiated
- Some work cannot be billed if PS made an error or was inefficient through no fault of the client; this is rare on the current team
- Time management and adaptability are critical – PS engineers often juggle five or six projects simultaneously, switching between them based on client responsiveness and urgency

Working Across Time Zones

- PS works with international clients across all time zones, including the Middle East
- Training must occur during the client's business hours, which can mean very early or very late hours for the PS team member
- Example – a client in Saudi Arabia had no business hour overlap with the US team at all

AI and the Future of PS Work

- Some campuses are turning to AI directly for report writing conversions instead of contacting PS first
- Training and consulting are seen as harder to replace with AI, since effective human-led training is currently more valuable than non-human training
- AI is expected to supplement engineering work, especially accelerating report conversions and migrations during legacy tool transformations, freeing engineers to focus on building new capabilities
- Higher ed institutions tend to be slower adopters of new technology (eg the slow adoption of SaaS); AI adoption is expected to follow a similar slower curve
- Security and data integrity concerns are significant in higher ed AI adoption, including how institutional data is shared with AI tools
- AI tools already help PS engineers write and convert SQL queries more efficiently
- Example – a school used AI to convert over 600 reports from Crystal Reports/Power BI to Argos, completing more than half quickly despite a small IT team

Partnerships

Speaker Background

- James Hawking – VP of Strategic Partnerships at Evisions
- Career path: healthcare software (support, then management) → finance within the same company (accounts receivable/payable, royalty payments) → financial services software (account management for equity compensation tracking tools) → Evisions, approximately 2.5 years ago
- Joined Evisions to help build out a formal partnership program alongside Brian Redeker (now

VP of Sales)

Why a Partnerships Program Exists

- **Strategic growth** – grow revenue and expand product offerings by increasing exposure to customers in the marketplace through partners
- **Cross-functional alignment** – without a centralized partnerships function, different departments (sales, engineering, support) could independently pursue conflicting or redundant partnerships, creating confusion and competing priorities
- The partnerships team acts as connective tissue/liaison between Evisions departments and individual partners
- **Lifecycle accountability** – evaluating and signing a partnership is the easy and exciting part; ensuring the partnership actually delivers value (new markets, real product value to customers) is the harder, ongoing work

Reporting Structure

- At Evisions, partnerships reports into the Chief Revenue Officer, alongside sales and customer success
- This is common but not universal – partnerships can also report into finance (financial perspective) or product (when the partner's technology is core to the product's functionality and affects pricing, markets, and regional availability)

Partnership Roles

- **VP of Strategic Partnerships** – oversees all partner relationships and tracks growth; splits time between administering existing partners and identifying/expanding new partnerships; serves as the escalation point for partner issues; works closely with the Chief Revenue Officer (budgeting and revenue targets), finance (budgets), and legal (contract review)
- **Partner Relationship Manager** – not currently filled at Evisions; would own a day-to-day relationship with a specific high-activity partner (eg a growing Middle East reseller relationship), monitor SLAs, and serve as first line of defense for issues

Partner Identification Process

- **Internal perspective** – identify needs based on existing products needing visibility or new products needing a component; framed as a buy versus build versus partner decision
- Partnering is a hybrid of buy and build – leverages an existing, proven technology without the full cost of acquisition; involves a financial arrangement such as revenue share (reselling) or a flat licensing fee
- **Market perspective** – partnerships team monitors the marketplace (trade shows, trade press, customer relationships) to understand the ecosystem and identify potential matches
- Partners can also approach Evisions directly, or internal teams can flag existing products that interact well with Evisions tools

Partner Evaluation Criteria

- **Product relevance** – how closely the partner's product matches Evisions' needs and the higher ed market's needs (eg FERPA compliance awareness for a partner unfamiliar with education-specific regulations)
- **Revenue potential** – a good technical and cultural fit is not enough if there is no meaningful new revenue opportunity
 - Example – a procurement software company with nearly 100% client overlap with Evisions was a good product fit but offered little new revenue potential, so the partnership

did not move forward

- Conversely, low client overlap plus strong technical fit can create a compelling case even if it requires investment

- **Partner readiness** – assesses whether the potential partner has the maturity to support Evisions clients (documentation quality, technical standards, support capability); described as more art than science

Technology Connection Evaluation Example (Hosting Provider for Maps/Argos)

- Engineering is brought in early to assess technical fit – can the provider meet Maps/Argos environment requirements
- Cost evaluation – is the provider’s pricing within a range that allows Evisions to mark up and resell profitably
- Security evaluation – new/unproven providers may offer attractive pricing but lack the reputation and track record needed for reliability
- Marketing alignment – both companies’ market positioning (eg high-quality vs bargain-basement) must be compatible to avoid messaging conflicts

Partner Approval and Implementation Flow

- Identification → evaluation (with input from relevant departments) → documentation of risks and mitigation steps → executive team review and approval → implementation
- Department feedback does not require 100% consensus, but concerns must be documented along with mitigation plans
- **Implementation** includes internal announcements, building any required integrations, and establishing monitoring (especially for previously flagged risk areas)
- Post-launch, the partnership is tracked against its original goals (eg access to a new market segment); both underperformance and overperformance should be evaluated and documented

Case Study – Invoke Learning (Reseller Partnership)

- Founded by Brendan Aldridge and Lige Hensley; approximately 15–20 employees
- Lige Hensley was previously CIO at Ivy Tech, Indiana’s largest community college system (over 100,000 students)
- **Problem identified** – campus data is fragmented across many systems (SIS, financial, LMS, security, meal card, etc), each with different definitions, timing, and language, making cross-system reporting inconsistent
- **Invoke Clarity** – data warehouse/lakehouse that pulls and normalizes data from 100+ higher ed system connectors (eg Ellucian Banner, Workday) into a consistent format
- **Invoke Unify** – builds large wide tables from the normalized data for reporting
- Invoke Learning is output agnostic – they aggregate and normalize data but do not provide reporting output themselves
- **Partnership structure** – reseller relationship; Evisions connects Argos directly to Invoke’s normalized data, providing clients a full data warehouse plus reporting solution

Reseller Relationship Mechanics

- **Technology partnership** – using a piece of a partner’s technology to provide Evisions’ solution
- **Reseller partnership** – Evisions can sell the partner’s product directly to clients and contract for it directly, with the partner providing support
- Pricing example – if Invoke’s retail price is \$100,000 and Invoke charges Evisions \$60,000, Evisions sells at \$100,000 retail and retains the \$40,000 (40%) margin as revenue

- **Benefits to the client of buying through the reseller relationship instead of separately:**
 - Lower combined sales cost typically results in a more competitive price
 - Single contract instead of two, reducing legal and committee review overhead
 - Better-tuned integration between the two products since Evisions configures specifically for Invoke Learning
 - Access to prebuilt data blocks that Evisions has already built for the Invoke data set and shared via the community

Ongoing Reseller Operations

- **Go to market** – Evisions sells the combined product directly; Invoke subject matter experts can be brought in for deep technical questions
- **Support** – Evisions generally supports the Argos piece independently due to familiarity with the tuned integration; Invoke is pulled in for elaborate/technical issues, with standard communication channels in place
- **Product integration** – prebuilt data blocks already exist for the Invoke data set; the product team factors Invoke compatibility into prioritization of new capabilities
- **Onboarding and training** – streamlined due to the frequency of working together

Cross-Departmental Involvement in Partnerships

- **Sales** – partnerships influences but does not control sales; provides targets, partner resources, documentation, and training, and explains how partnerships help sales hit their goals
- **Support** – ensures documentation and team training are in place to reduce support incidents; strong inter-team relationships allow quick resolution when issues arise
- **Engineering** – builds required connectivity; partnerships relies on engineering’s technical guidance for SLAs, server capacity, and proper support levels, since partnerships staff are not deep technologists
- **Marketing** – coordinates joint announcements and campaigns so that neither company’s marketing team is overburdened, and to bring fresh messaging perspectives
- **Finance** – budgets and revenue projections, pro forma modeling
- **Legal** – builds contracts that protect both parties when things go wrong, even though early-stage excitement makes this feel unnecessary
- **QA** – tests interactions with partner systems, since partner systems may only provide partial pieces of functionality that need to be validated together

Why Contracts and Termination Clauses Matter

- Example scenario (prior company, financial services) – a major white-label partnership was signed with carefully negotiated termination clauses, despite initial reluctance since both sides expected long-term success
- Six months after signing, the company providing the technology was acquired by a direct competitor of the partner using that technology, creating a conflict neither side anticipated
- Because termination, transition periods, and pricing changes had already been negotiated and agreed upon, both sides were able to navigate the situation in an orderly, civil way
- Lesson – detailed contract language, even seemingly minor clauses, can have major consequences depending on how circumstances change

Argos Workshop Overview

Trainer Background

- Gail is leading the Argos training, Tuesday through Thursday this week, following the introduction from PS
- Approximately 11.5 years at Evisions, hundreds (likely thousands) of clients trained
- Background note – entered the technology field unexpectedly from a non-technical academic background, illustrating that career paths in tech are often nonlinear

Why Training Happens

- New client implementing Argos for the first time
- Existing client whose originally trained staff have turned over and need to retrain a critical mass of new employees
- Individual new hires sent for training as needed
- Specialized training requests (eg how to build charts, or training specifically on Argos X)

Training vs Development

- Training teaches how to use the tool; it does not build the client's specific end deliverables
- Example – a project manager wanted to skip foundational tool training and jump straight to building needed data blocks, but the training's purpose is tool literacy, not deliverable creation
- Goal of training is to ensure the client knows where to find resources and how to solve problems independently afterward

Pre-Training Preparation

- A client call is held in advance to gather background information:
 - Is the client new to their ERP, or have they been using it for years
 - Are they implementing Argos at the same time as a new ERP (not recommended, but sometimes unavoidable)
 - Who will attend, and what is their technical/institutional background
- **Training requirements** – Windows required, dual monitors recommended, a software launcher must be downloaded (this is flagged in advance since some institutions restrict software downloads)
- Training is sometimes delivered in Evisions' test environment, and sometimes in the client's own environment for better data relevance; this may require advance setup and adjustments to exercises

Training Delivery Format

- Training materials are reference guides, not strict step-by-step instructions; the live instructor is essential
- New concepts are introduced incrementally, then reinforced and repeated before adding more
- PowerPoint presentations are shared with attendees after sessions
- Trainers direct clients to Evisions support resources for ongoing help; the trainer is not a long-term support contact
- Delivery format has shifted from primarily in-person (pre-COVID, three full days Tuesday through Thursday) to primarily remote over Zoom, with experimentation in session length (eg a newer four-hour "one-on-one training" format)

Training Environment Setup

- Training environment URL: training.evisions.com

- Username format: first name underscore last name (not case sensitive)
- Initial password: evisions2026 (all lowercase, case sensitive), with a forced password change on first login
- Environment is available throughout the summer but goes offline overnight and on weekends
- Within the client, the **Argos** button launches the Argos client, which may require downloading the Evisions Application Launcher on first use

Argos Product Overview

- Argos client (desktop application) is required to build data blocks
- Argos WebViewer was introduced after the original client
- Argos X was introduced roughly a year ago, with the tagline "go from web viewing to web doing"
- Argos is Evisions' flagship product and main revenue driver

Default Roles in Argos

- **Report Viewer** – end user role; can view and run reports only
- **Report Writer** – includes report viewer permissions plus the ability to create reports
- **Data Block Designer** – includes both other roles; full permissions including building data blocks
- Role determines which columns/permissions a user sees in the Argos interface

Data Block Structure

- A **data block** is the foundation of Argos reporting and has two parts:
 - **Dashboard** – the visible part; where end users select parameters and can view returned data; starts as a blank gray canvas that the designer fully customizes (colors, images, layout, parameters)
 - **Report query** – the hidden part; a SQL statement that pulls the data used for formatted reports; not visible to end users
- Two ways to build a report query:
 - **Visual design tool** – drag in tables, join them, and select fields visually
 - **Direct SQL** – paste an existing SQL statement as is, as long as syntax matches
- The report query is not strictly required; if no formatted output is needed, the dashboard alone can return data
- Dashboard and report query communicate – parameters selected on the dashboard (eg a term) can be passed to the report query to filter results

Reports

- A **report** takes the report query output from a data block and formats it into a selected output type
- Four report types exist, each with its own interface, selected based on the desired output format

Dashboard Objects and Parameters

- **Image** – static visual element (eg logo or branding); must be pre-sized before import, since Argos does not edit images; ideal height is roughly 100–150 pixels
- **Label** – text element with configurable font, color, size, and styling (bold, underline, italic)
- **Edit box** – free-text input field for end users; useful when a dropdown would be impractical (eg searching by name, where a dropdown list is capped at 1,000 items)
- **Drop-down box** – single-select parameter; allows the user to choose only one value at a

time

- **List box** – can be configured for single-select or multi-select; multi-select allows users to choose multiple rows at once
- **Multi-column list box** – displays multiple rows and columns; selecting a row selects the entire row, not individual columns; often used for display or as an intermediary selection step
- **Panel** – container used to group and organize objects; supports a library of reusable saved objects (eg branded layouts, common parameters) for consistency across data blocks

SQL Behind Dashboard Objects

- Parameter objects with dynamic data (eg dropdowns, list boxes) typically have their own SQL statements behind them to filter available options (eg only show terms with data, or only show schools/colleges with data for the selected term)
- A single dashboard can have multiple SQL queries powering different parameters
- The report query (within the data block, not the dashboard) is separate and is what powers the actual report output

Designing for Error Prevention

- Dashboards should be designed so end users cannot select impossible or invalid parameter combinations
- Rationale – unconstrained free-text or unclear parameters lead to user errors, frustration, and increased support burden on the designer
- Best practice – use dynamic, constrained parameters (dropdowns, filtered list boxes) wherever possible instead of unrestricted text entry

Summary of Roles and Data Block Components

- Three roles: Report Viewer → Report Writer → Data Block Designer (each higher role includes the permissions of the ones below it)
- Data Block Designer builds data blocks, which consist of a dashboard (visible) and a report query (hidden)
- Report Writer creates reports, which format report query output into a chosen output type

First Hands-On Exercise – Person Search Data Block

- **Goal** – build a simple data block that searches for a person by first name and returns their ID, full name, email address, and mailing address
- **Design considerations for the dashboard:**
 - Decide on branding – colors, images, layout (can be customized per institution or kept simple for training purposes)
 - Add a label/title for the search
 - Add an edit box parameter for the first name search, since a dropdown of all first names would be impractical and exceeds the 1,000-item limit
 - Add instructional text as needed
- **Steps to create a data block:**
 - Navigate to the correct personal folder first (working in another user's folder is restricted)
 - Add a data block via the Explorer toolbar icon (three cubes with a plus symbol) or by right-clicking and selecting New → Data Block
 - Rename the data block descriptively (eg "Person Search")
- **Connection setup:**
 - Connections are created in Maps, the underlying data administration layer

- For this exercise, use the existing "Demo 17" connection under Banner databases
- Leaving the connection blank causes Argos to prompt for a connection every time a query runs
- Connections can be changed later (eg moving from a test environment to production once a data block is validated)

- **Description field:**

- Every data block should have a description, even though it is optional
- Descriptions are searchable via Argos' built-in search (which searches object names and descriptions)
- Descriptions help future users and the original designer understand the purpose of a data block, especially as the number of data blocks grows
- Best practice – always fill in the description at creation time, since retrofitting descriptions later is much harder

Homework / Next Steps

- Optional – prepare an image for the dashboard (correctly sized, since Argos cannot resize images); schools logos are available, or any custom image can be used
- If using a custom color, obtain the RGB value of that color (standard colors like black and white are already available; custom shades require RGB lookup tools)
- Next session – begin building the dashboard for the Person Search data block using the edit button

6/16 – Argos Workshop Basic Training & Data Engineering

Argos Workshop Basic Training

Designing the Dashboard

- Dashboard starts as a blank gray canvas; all visual elements are placed manually by the data block designer
- **Panel** – container object used as the foundation of the dashboard; analogous to a picnic blanket: objects placed on a panel move together as a group; enables saving and reuse as a library template
- Two-panel structure recommended for branding:
 - Outer (background) panel – set to institution or custom color; defines the overall frame
 - Inner (content) panel – set to white or neutral; placed on top of the background panel; holds all labels, parameters, and images
- Panel properties are set via the left-side properties tab (color, height, width, font defaults); font set on a panel is inherited by objects placed on it
- Custom colors require an RGB value entered via the custom color picker; standard colors (white, black, etc) are pre-available
- Objects in the library can be retrieved and reused across data blocks; saves rebuild time and standardizes look and feel across dashboards

Dashboard Objects

- **Image** – placed via the landscape icon; three source options: local file, server URL (useful when logos are updated frequently), or a SQL variable; must be pre-resized before import (ideal height 100–150 pixels); auto-size property must be set to yes to display correctly
- **Label (A icon)** – text element; font, size, color, bold/italic/underline all configurable; text

content edited via double-click or the properties tab; accessible text field available for screen reader support

- **Edit box (AB cursor icon)** – free-text input for the end user; used when a dropdown is impractical (eg searching by name across thousands of values); description property displays as a hover hint at runtime
- **Drop-down box** – single-select parameter; can be configured to auto-select the first value so the user does not have to interact with it manually
- **List box** – single or multi-select parameter; multi-select enabled via the multi-select property; displays data from a SQL statement or manual entry
- **Multi-column list box** – displays multiple rows and columns; selecting a row selects the entire row
- **Checkbox** – used for boolean (yes/no) data

Alignment and Layout

- Objects can be aligned via mouse drag, property values (left, top, height, width), or the alignment toolbar
- Alignment toolbar: selecting one object gives horizontal/vertical center options relative to its parent panel; selecting multiple objects (shift-click) enables align by left, right, top, bottom, or center
- The first object selected in a multi-select alignment becomes the reference; subsequent objects align to it
- Keyboard shortcut: control + arrow keys nudge selected objects by small increments

Naming Variables

- Every parameter object (edit box, list box, dropdown) gets a system-generated default variable name (eg edit1, dropdown1); these must be renamed immediately
- Variables tab (under form design) shows all referenceable objects on the dashboard
- Naming convention: form name, object type abbreviation, description (eg name_EB_first_name for an edit box capturing first name on the main form)
- Variable names must match exactly when referenced in the report query; renaming a variable after it has been referenced in a query will break the connection

Report Query – Visual Design Tool

- **Visual design tool** – graphical interface for writing the SQL report query without typing raw SQL; tables are added, fields are selected from a dropdown, joins are created visually; recommended for training since it avoids typo-related errors
- **SQL free type** – alternative mode where a raw SQL statement is typed or pasted directly; used when a pre-written SQL statement already exists; does not provide field autocomplete
- **Adding tables** – use the Add Table button; include the schema prefix (eg saturn.spridem) to bypass the table selection prompt
- **Selecting fields** – double-click a field in the table to move it to the select list, or drag and drop; selecting via the table/field dropdowns does not pull in data dictionary aliases and is not recommended
- **Calculated fields** – set the table dropdown to "calculated" to write a subquery or concatenation as a field; used for scalar sub-queries (eg pulling a single email address) or combining fields (eg org code concatenated with org title for display)
- **Distinct** – toggle available in the visual design tool to prevent duplicate rows when joining tables with one-to-many relationships
- **Where clause (conditional fields)** – connects dashboard parameters to the query; field

selected, operator set, and value references a dashboard variable via the ABC button; functions like UPPER() can be applied to make matching case-insensitive; wildcards (%) used with LIKE for partial text matching

- **Oracle concatenation** – double pipe characters (||) join strings; single quotes wrap literal text or wildcards (eg 'GA'% returns all values starting with GA)
- **Order by tab** – sets the sort order of the returned data; defaults to the order defined in the report query; can be overridden at the report level
- **View SQL button** – converts the visual design output into a readable SQL statement for review; changes made in view SQL mode switch the query to free type mode permanently

Data Dictionary

- **Definition** – a metadata layer maintained in Maps that provides pre-configured table joins, field aliases, and default filters for common Banner tables
- Only applies when using the visual design tool, not SQL free type
- Visual indicators: green cubes and green join lines indicate data dictionary-sourced configuration; blue join lines indicate user-created joins
- Examples of data dictionary behavior:
 - Spriden change indicator is null – automatically filters to the current name and ID record, preventing duplicate rows from name change history
 - FTBORG next change date – automatically retrieves the most current department name without requiring a manual where clause condition
 - Pre-joined tables – some table pairs are already joined in the data dictionary and do not require a manual join step
- Contributions to the data dictionary – a user can right-click a join line and select "contribute"; the suggestion goes to a Maps administrator for approval before becoming part of the dictionary

List Box and Drop-Down SQL

- Parameter objects that display data (list boxes, dropdowns) each have their own SQL statement behind them to populate the options
- **SQL statement option** – dynamically pulls values from the database; recommended when the list changes over time (eg new departments or employee classes) or when values should only appear if they have associated data
- **Manual entry option** – a static list typed directly into the interface; recommended for short, stable lists (eg two address types: permanent and mailing)
- When using SQL for a list box or dropdown:
 - The visual design tool is opened via the pencil and ruler icon
 - The field to display to the user is selected at the finish step (separate from what is stored and passed to the report query)
 - Two fields are often selected: a code (passed to the query) and a description (shown to the user)
- **Display field concatenation** – when a list box can only show one field, the code and description can be concatenated into a single calculated field using double pipes so both are visible to the user
- **Auto-select** – property on dropdown objects that auto-selects the first value at runtime; prevents the user from accidentally leaving the parameter blank
- Multi-select is enabled per list box via the multi-select property (default is no); applies to individual list boxes, not to dropdowns

Reports – Types and Creation

- A report is created under a data block and formats the report query output into a selected output type
- Report type must be selected first and cannot be changed after creation; a new report must be created if the wrong type is chosen
- **Four report types:**
 - **CSV** – simplest to create; opens in Excel; best for getting raw data out for further manipulation; does not support row-level totals or section formatting
 - **Extract report** – three sub-types (fixed-width, delimited, XML); delimited version offers formatting options not available in CSV (eg prefix/postfix per field, section-level totals); used when data must be imported into another system or when a formatted summary row is needed
 - **Banded report** – most fully formatted; typically output as PDF; supports images, colors, fonts, and precise layout control; used when output must match an exact visual specification
 - **Crosstab** – similar to a pivot table; displays summarized data without row-level detail; used when only aggregate counts or sums are needed
- Report creation flow: select data block → report writer actions → new → select type → name and describe → create → design (select and order fields) → set sort → apply/ok
- **Sort override** – the sort tab allows the report-level sort to override the report query order; check the override box and select fields to sort on
- **Shared vs private** – reports default to shared (visible to all users with folder access); private restricts visibility to the creator only; administrators can see all regardless
- Running a report in the Argos client: report viewer actions → run report → use the gear icon to save/open the output file
- **Save current settings** – allows parameter selections to be saved and recalled later; eliminates the need to re-select parameters on every test run during development

Extract Delimited Report – Formatting Fields

- CSV files opened in Excel may strip leading zeros from numeric-looking values (eg student IDs, zip codes, social security numbers) or prompt a "convert / do not convert" dialog
- In an extract delimited report, each field in the detail section has a **prefix** and **postfix** property
- Setting prefix to "=" and postfix to "\"" wraps the value in Excel-safe text notation, preventing leading zero stripping and eliminating the convert prompt
- Apply this to any field that starts with zeros, contains mixed alpha-numeric values, or has inconsistent alignment due to Excel's auto-type detection
- Do not apply prefix/postfix formatting to fields where arithmetic will be performed; the formatting converts the value to text and breaks numeric formulas
- Section-level totals (sum of columns) can be added to extract delimited reports via a summary section; note that once the file is in Excel, the total is a static value and will not update if rows are added or removed

Argos X vs Argos Client

- Argos client is required to build and edit data blocks and reports; most end users will not use the client
- Argos X is the primary interface for end users to run reports and explore data
- Argos X differences from the client:

- Run report downloads output directly without the three-option save/email/gear prompt
- **Explore report data** – built-in ad hoc analysis tool; users can filter, group, and pivot the report query results; data can be exported as CSV or Excel from this view
- Dark mode and custom themes available per user preference

Support and Resources

- **Built-in help** – F1 or the help button within Argos; context-sensitive, opens to the topic relevant to the current screen; searchable
- **User guides** – three role-specific guides (data block designer, report writer, report viewer); PDF and web versions available; web version is more current; accessed via Argos support resources within the help system
- **evisions.com** – public resources including webinars, on-demand webinar recordings, user groups (regional and role-based, meeting quarterly), e-studies (institution case studies), blog, and event listings (Cohesion fall conference, spring conference adjacent to Ellucian Live)
- **Support and community site** – requires login; for creating support cases with defined SLAs by severity; also hosts product topic forums where the community and Evisions staff answer questions
- **On-demand training videos** – available via the support and community site; cover core Argos tasks; useful for reviewing concepts after a training session
- **CO-OP (community data block library)** – data blocks created by other institutions can be downloaded and imported; filtered by product, SIS platform, and functional area; useful starting points that can be customized to match local data and branding
- Data blocks are exportable (Export button in the Argos client) as files that can be shared or uploaded to the CO-OP

Library Objects

- Objects or groups of objects saved from a dashboard can be stored in the library for reuse
- **Book with plus icon** – adds an object to the library
- **Book without plus icon** – retrieves an object from the library
- Library is scoped to: personal (username folder) or all users (shared across all designers)
- Common library uses: branded dashboard templates (background panel, white content panel, logo, dynamic title), pre-built parameter objects (eg term dropdowns), reusable layout structures
- **Dynamic data block name label** – a label can be made data-aware (property: data aware = yes) and bound to the system variable `$datablock.name`; the label then automatically displays the name of the current data block at runtime without manual text entry

Key Terms – Argos

- **Data Block** – the core Argos object; contains a dashboard (visible) and a report query (hidden); created by data block designers
- **Dashboard** – the user-facing part of a data block; where parameters are selected and data can optionally be displayed
- **Report Query** – the SQL layer behind the dashboard that retrieves data used to build reports
- **Panel** – a container object used to group and move dashboard elements together; also the unit saved to and retrieved from the library
- **Edit Box** – free-text input parameter on a dashboard
- **Scalar Sub-query** – a self-contained SQL query embedded within a select statement to retrieve a single value from a separate table (eg pulling a preferred email address)

- **PIDM** – internal Banner ID that links a person across all Banner tables; used as the join key between tables in many Banner queries
- **Spriden** – Banner table storing person name and ID information; change indicator field used to filter to the current name record
- **Inner Join** – join type requiring a matching record in both tables; omits rows where a match does not exist in one table
- **Outer Join** – join type that includes all rows from one table even when no matching record exists in the other; used when data in the joined table is optional (eg email address)
- **Prefix / Postfix** – field-level formatting properties in extract delimited reports used to wrap values in Excel-safe text notation and prevent leading zero stripping
- **CO-OP** – Evisions community library where institutions share data blocks for others to download and adapt

Data Engineering

Team Introduction

- Data engineering team at Evisions: Amit, Chris, Brian
- The team sits between the PS team (client-facing, report-building) and software engineering (product development); they handle technically demanding data work that falls outside the wheelhouse of either
- Primary work: building ETL pipeline tools, bridging client asks with data engineering solutions, and building internal tools PS can use

ETL Pipeline

- **ETL (Extract, Transform, Load)** – the foundational concept of data engineering; the three-step process of moving and reshaping data across systems
 - **Extract** – pull raw data from one or more source systems (eg LMS, SIS, financial system)
 - **Transform** – clean, normalize, join, and reshape the data (eg standardizing date formats, joining student records from two systems using matching keys, filtering to active records only)
 - **Load** – write the transformed data to a target destination (eg a data warehouse, another database, or a report output)
- ETL is the industry standard term for data pipelines; also referred to as workflows in many tools
- Argos can only report against one database at a time; ETL pipelines solve this by consolidating multiple source databases into one unified data lake or warehouse that Argos can then query

Data Silos in Higher Ed

- Higher ed campuses typically have many separate systems (SIS, LMS, financial, housing, dining, building access, meal card, ID card, career services, etc), each with its own data, naming conventions, timing, and definition of a student
- Cross-system reporting is impossible without normalizing and consolidating this data first
- Higher ed has historically been slower to adopt cloud and modern data infrastructure compared to other industries, though that is now changing
- Institutions are increasingly underfunded and understaffed; the ability to hire dedicated data engineers is limited
- Evisions' data engineering team packages pre-built pipelines and reports so institutions can access modern data capabilities without needing internal data engineering talent

Catalyst – Evisions ETL Pipeline Tool

- **Catalyst** – an internal Evisions pipeline tool under development; designed to give clients a guided, lower-effort path to consolidating fragmented campus data
- Core pipeline flow:
 - **Extraction** – targets a data source (eg Canvas LMS), pulls raw data, and moves it into a data lake; one or many sources can be extracted
 - **Data lake** – a single storage location for raw, unprocessed data from multiple sources
 - **Transformation** – takes data from the lake, cleans and restructures it for reporting purposes; scoped to only the fields and windows of data relevant to a specific report; improves downstream query performance
 - **Data warehouse** – the output of the transformation layer; structured, cleaned, and optimized for reporting; functions more like a data mart (solution-focused, not a general-purpose warehouse)
- Evisions currently has two packaged pipelines with two corresponding reports; the goal is to build a full suite where institutions pay a one-time setup fee and then add new solutions at low marginal cost
- Catalyst features: named extractions, transformation builder, model management, job scheduling, job history/logging, connection configuration

DBT (Data Build Tool)

- **DBT** – an industry-standard open-source tool for managing SQL-based data transformations at scale
- Used by Evisions within Catalyst for the transformation layer
- Key concepts:
 - **Models** – individual execution nodes; a single SQL transformation step; many models chain together to form one complete transformation pipeline
 - **Materialized views** – intermediate data states created along the pipeline; reusable by multiple downstream models so the same query does not need to be rerun from source each time
 - **Dependencies** – DBT manages the order in which models run based on what each model references
 - **Common cohort definitions** – institutions define a cohort (eg enrolled students for a given term) once as a model; all downstream reports reference that model instead of re-querying from source
- DBT addresses the problem of complex pipelines where many transformations share intermediate states; it provides version control, dependency management, and documentation for those transformations

Other ETL Tools

- Many commercial and open-source tools exist for building ETL pipelines; most offer visual drag-and-drop workflow builders
- **Workato** – integration platform as a service; connector-based (Salesforce, Slack, etc); used widely in cloud ETL; previously used by a team member
- **Zapier** – another workflow/integration platform; similar connector-based model
- **Alteryx** – visual, SQL-based data preparation and blending tool; drag-and-drop workflow including joins, filters, aggregations, and predictive analytics steps; outputs to CSV, database, or other targets
- **n8n** – open-source visual workflow builder; increasingly AI-driven; node-based with support

for AI agents and database outputs

- General pattern across tools: input data → transform/blend steps → output destination; referred to as workflows, pipelines, or ETL depending on context

Case Study – SUNY Transcript Pipeline

- Client: SUNY (State University of New York system)
- Problem: SUNY receives approximately 200,000 high school transcripts per year and wants to analyze 30,000 of them to match students to college admission criteria; transcripts arrive as unstructured PDFs
- **Pipeline built by the data engineering team:**
 - **Input** – PDF transcript file received; file name, applicant ID, and receipt date logged
 - **OCR (Optical Character Recognition)** – extracts all raw text from the PDF; cost per run: \$0.05
 - **AI normalization** – sends extracted text to an AI model; model structures the text into a normalized class list; cost per run: \$0.15
 - **Derived values** – applies formulas to the class list to calculate GPA by subject area, years of coursework per subject, specific course grades, and SAT/ACT scores; cost per run: \$0.0037
 - **Confidence scoring** – flags records that require manual human review based on data quality signals
 - **Output** – structured database record ready for downstream use (eg applying Cornell's math requirement: 3+ years of math, 95+ GPA)
- Demonstrates a complete ETL pipeline: extract (OCR), transform (AI normalization + derived calculations), load (structured database output)
- Tech stack: TypeScript, Vercel hosting, Vercel AI SDK (`generateText` method), Vercel workflows (pipeline orchestration with automatic retry logic and step-level status logging)

Key Terms – Data Engineering

- **ETL (Extract, Transform, Load)** – the three-step process of moving and reshaping data from source systems into a unified target; the foundational concept of data engineering
- **Data Lake** – raw storage for extracted data from one or many sources before transformation
- **Data Warehouse** – structured, transformed storage optimized for reporting; downstream of the data lake
- **Data Mart** – a solution-focused subset of a data warehouse scoped to a specific reporting need; what Catalyst's warehouse layer functions as in practice
- **DBT (Data Build Tool)** – open-source SQL transformation management tool; handles model dependencies, materialized views, and pipeline orchestration
- **Model (DBT)** – a single SQL transformation node within a DBT pipeline
- **Materialized View** – an intermediate data state stored and referenceable by multiple downstream models; avoids redundant re-querying from source
- **OCR (Optical Character Recognition)** – technology that extracts text from images or PDFs
- **Connector** – a pre-built integration between an ETL tool and a specific data source (eg Salesforce, Slack, Banner)
- **Workflow** – a general term used across ETL tools for a configured pipeline of steps that process and move data; synonymous with pipeline in most contexts
- **Vercel AI SDK** – a TypeScript package from Vercel providing simplified methods for calling AI models (eg `generateText`) within web applications

- **Vercel Workflows** – a Vercel-native pipeline orchestration layer; provides automatic retry logic, step-level status tracking, and timing data for pipeline runs

6/17 – Argos Workshop Advanced Training

Argos Workshop 3 – Advanced Dashboard Techniques

Address List for Employees Data Block – Continued

- Continued building the address list for employees data block started in a prior session
- Dashboard parameters: employee class list box, department list box, address type dropdown (manual entry)
- Report query tables: Spriden, Spraddr, Pibempl, Ptreecls, Ftborg – almost all joins pre-configured by the data dictionary; only the Pibempl-to-Ptreecls join required manual creation (on e-class code)

Selecting Fields in the Report Query

- Fields selected from each table via double-click or drag-and-drop into the select list
- **From Spriden** – PIDM (internal ID), ID (Banner ID), last name, first name, middle name
- **From Spraddr** – address type code, street line 1, street line 2, city, state, zip code
- **From Pibempl** – employee class code, current hire date
- **From Ptreecls** – employee class long description
- **From Ftborg** – org code, org title (department)
- Field order within the select list does not affect query correctness but choosing a logical order makes the data easier to navigate later; analogy: keeping bills organized by denomination vs wadded up – both have the same money, but one is easier to use
- Fields can be repositioned: double-clicking a field appends it to the end; clicking and dragging allows precise placement

Calculated Field – Years Worked

- Table dropdown set to "calculated" to create a field not sourced from a table
- Default name overridden with a descriptive alias (eg `years_worked`)
- Formula: `(sysdate - pibempl_hire_date) / 365.25` – calculates years of service based on the difference between today and the hire date
- **Key concept** – age and tenure are moving targets; they are not stored as static values in Banner; they must be calculated at query time using `sysdate` (Oracle's current date function)

Where Clause – Connecting Dashboard Parameters

- **Active employees filter** – Pibempl employee status field, condition: 'A' (single quote, capital A)
- **Department list box** – Ftborg org code connected to the department list box variable via the ABC button; equals relationship handles multi-select automatically behind the scenes
- **Employee class list box** – Pibempl e-class code connected to the employee class list box variable; same pattern
- **Address type dropdown** – Spraddr address type code connected to the address type dropdown variable
- **Multi-select and equals** – when a list box has multi-select enabled, Argos internally converts the equals condition to an IN clause; the designer does not need to write IN manually

Aliases and Column Headers

- The "as" alias in the select list becomes the column header in the output

- Renaming aliases before finishing the query saves cleanup time later (eg renaming the auto-generated alias to "Department Code" or "Department Description")
- Aliases can be cleaned up at the report design step as well, but doing it early is more efficient

Revision Control

- **Definition** – version history for a data block; allows reverting to a prior saved state if changes break the query
- Enabled per data block via right-click → Security → Revision Settings → Yes; number of versions stored is configurable (eg 10)
- Every save after enabling revision control requires a note and marking the version as active; the note field must have content to hit OK
- **Trade-off** – enabling revision control early can be annoying if the designer saves frequently during active development; best practice is to enable it once the data block is stable and changes are less frequent
- **Manage revisions** – accessible via right-click on the data block; shows all saved versions with their notes; clicking the star on a prior version reverts to that state
- **Regular save vs revision save** – there is a "save regular" option that saves without creating a new revision; using this habitually defeats the purpose of revision control

Union Queries – Select All via Checkbox

- **Use case** – end users frequently want to select all values from a list box without clicking through hundreds of options; a checkbox can trigger an "all selected" state automatically
- **Four-step process:**
 - **Step 1 – Create checkbox** – add checkbox object to the dashboard; rename the variable (eg `main_CB_all_eclass`); set text label (eg "Select All"); set checked = yes (defaults to checked); change checked value from 1 to Y and unchecked value from null to N for readability in SQL
 - **Step 2 – Modify the list box SQL** – add a where clause condition to the list box's SQL: if the checkbox is unchecked (N), return the normal list; add a union query that returns a single synthetic row (eg code XX, description "All Employee Classes Selected") when the checkbox is checked (Y); Oracle requires `sys.dual` as a dummy table for union queries with no real source table
 - **Step 3 – Auto-select the list box** – set the list box's auto-select property to yes so the synthetic "all selected" row is highlighted without requiring the user to click
 - **Step 4 – Update the report query where clause** – replace the simple equals condition with a conditional group containing an OR: either the e-class code equals the list box selection, OR the checkbox is checked (Y); the OR must be wrapped in a conditional group (use the "add conditional group" plus button in the visual design tool) to ensure correct SQL parenthesization
- **OR parenthesization** – SQL without parentheses around an OR can misread the entire condition; the visual design tool adds parentheses automatically when a conditional group is used; analogy: a restaurant week deal offering an appetizer, entree, dessert, and "coffee or tea" – without parentheses, a computer might interpret "tea" as the only item ordered
- **Case sensitivity** – checked/unchecked values and where clause conditions must match exactly (eg capital Y vs lowercase y will cause no results to return); computers are fully literal

Union Queries – Select All via All Option in List Box

- **Alternative to the checkbox approach** – adds "All [Items] Selected" as the first option in the list box itself rather than using a separate checkbox; three steps instead of four (no checkbox creation)
- **Sort order field** – a fourth calculated field (numeric) is added to the select: value 1 for the "all" synthetic row, value 2 for real rows; used in the order by to pin the "all" option at the top
- **Union query** – same `sys.dual` dummy table used; four fields selected to match the main query's field count; "all" row uses a fake code (eg XX) and a descriptive label (eg "All Departments Selected")
- **Order by** – must reference fields by position number (eg field 4 for sort order, field 1 for code) rather than by name when a union query is involved; names are not resolvable across union branches
- **Report query update** – same OR conditional group pattern as the checkbox approach; instead of checking the checkbox value, the OR condition checks whether the org code equals the fake XX code
- **Consistency recommendation** – pick one approach (checkbox or all-in-list) and stick with it across data blocks for a consistent end user experience; both accomplish the same goal

Lock/Edit Conflict Resolution

- Argos locks a data block when a user enters edit mode to prevent simultaneous conflicting edits
- If a user exits improperly (eg browser or app crash), Argos may retain the lock for several minutes
- **Workaround** – copy the data block and paste it in the same folder; work on the copy while the original releases its lock; delete the original once the lock clears
- Maps administrators can manually release locks
- **Trash bin note** – deleted data blocks go to a trash bin visible only to administrators; they are not permanently deleted immediately and can be recovered; regular users do not see the trash bin icon

Report Types Recap

- Three ways to create a new report: (1) report writer actions → new, (2) page-with-plus icon at top of column, (3) right-click → new → report
- Report type selection is permanent – if the wrong type is chosen, the report must be deleted and recreated
- **CSV** – created for the address list data block; simplest type; no section formatting

Argos Workshop 4 – Charts, Multi-Form Dashboards, and Extract Reports

Multi-Form Dashboards

- A data block dashboard can contain multiple forms, each displayed as a tab
- **Adding a form** – forms tab → click plus; rename the new form (eg "chart")
- **Tab configuration** – select a form → properties tab → tab section at bottom:
 - Tab visible: change to yes
 - Tab size: small, medium, or large (medium or large recommended; tabs render better in Argos X/WebView than in the client)
 - Tab name: what displays on the tab (eg "Home", "Chart")
 - Tab color and tab font color: customizable with RGB values; colors may not persist

overnight and should be noted separately (pink: 236, 67, 153)

- **Form order** – the first form in the list is where users land when they open the data block; reorder forms accordingly
- **Branding on secondary forms** – can reuse library templates; logos and data block name labels can be removed from secondary forms while keeping background colors, reducing visual clutter

Navigation Between Forms

- Three object types support the OnClick property for navigation: buttons, text labels, and images
- **Button** – added via the button object icon; variable name set in the variables tab (eg `main_BT_go_to_chart`); text property sets the button label; background color, font color, border color all configurable; cursor property can be set to hand pointer so hovering signals clickability
- **OnClick property** – set to "Activate Form"; drop down to select the target form; this is what moves the user from one form to another
- **Text label navigation** – same OnClick property available; convention is to style as blue underlined text and set cursor to hand pointer to mimic a hyperlink
- **Image navigation** – images also support OnClick; a chart icon from the library (all users → general dashboard options) can serve as a visual button to the chart form
- **Return navigation** – add a "Return Home" label on the secondary form with OnClick → Activate Form → main form; blue, underlined, bold, hand cursor
- Test navigation before building additional content; confirm clicking goes to the correct form and clicking return comes back

Charts – Overview

- Charts are added to a form via the pie chart icon in the toolbar
- **Standard vs Classic** – two chart types exist; classic charts predate the web viewer and may display inconsistently between client and web; standard charts are web-friendly and are the recommended choice; classic charts are retained for backwards compatibility
- Argos X has additional modernized chart options available in the dashboard designer

SQL Variable for Chart Data

- Charts require a dataset; the recommended approach is to create a SQL variable first and then attach it to the chart
- **Creating a SQL variable** – variables tab → plus button → SQL statement → OK → visual design tool
- Start by pasting (paste button, not Control+V) a copy of the main report query as a starting point
- **Modifications for the chart query:**
 - Enable summing (sigma button in the bottom left)
 - Delete unnecessary fields; keep only the ID field (to count) and the employee class description (to group by)
 - Set ID field summary to Count, rename alias to "employees"
 - Set employee class description summary to Group By, rename alias to "employee class"
 - Where clause: leave unchanged (same parameter logic as the main query); add one additional condition – the run chart button must not be null (is not null), preventing the chart from auto-running when the dashboard loads

- Order by: remove existing sort; replace with employee class description field (or field position 2)
- Validate the SQL variable when prompted; parameters can be left blank during validation (syntax is still checked)
- Rename the SQL variable after creation via right-click → rename (eg `SQL_chart`)

Chart Wizard

- **Series tab** – add a series; give it a name (eg "employee per e-class"); select dataset (the SQL variable); value field = the count field; label field = the grouping field (employee class)
- **Type tab** – choose chart type (bar chart default); types can be changed after creation; Argos X chart types are more modern
- **Labels tab** – toggle visible labels on bars (value or percent); percent useful for proportional comparisons
- **Bar options** – horizontal or vertical; color customizable (applies to all bars)
- **Axis options** – usually left at default; adjustable if data is highly skewed
- **Background panel** – cosmetic; leave at default
- **Title tab** – select "main title" row; check visible; enter title text (eg "Number of Employees per Employee Class"); set alignment and font
- After the wizard, resize the chart object by dragging; variable name can be set via the variables tab after creation

Visible and Enabled Properties

- **Visible** – controls whether an object is shown; can be set to a variable so visibility depends on whether that variable has a value
- **Use case for chart** – set the chart's visible property to the run chart button; the chart stays hidden until the user clicks run, preventing a "no data" message from appearing before the user is ready
- **Other uses for visible** – walking users through a dashboard step by step (show the next parameter only after the first is selected); showing/hiding help text via a help button
- **Enabled** – similar to visible but grays out the object rather than hiding it; most commonly used on buttons to prevent interaction until conditions are met (eg a preview button grayed out until a course is selected)
- Both properties can only reference a single variable; if visibility depends on multiple conditions, create a SQL single variable that evaluates the combined condition

Extract Fixed-Width Report – State Upload Exercise

- **Scenario** – HR needs to send employee addresses to the state government in a specified fixed-width text format; format is defined by a technical specification document
- **Fixed-width format indicators** – evenly spaced columns suggest fixed-width; a delimited format would use separators between fields
- **Report type selected** – extract text report (fixed-width); output as .txt
- **Sort order** – overridden at report level: zip code, then last name, then first name

Extract Report Sections

- Extract reports have six available section types: title, detail, summary, child, header, footer
- **Title** – appears once at the very top of the file; used for file-level metadata (eg institution name, contact email, run date)
- **Detail** – one row per record; the main body of the report
- **Summary** – appears once at the very bottom; used for totals or file-level footers

- **Child** – follows each row of its parent section; parent can only have one child, but children can have children (for multi-level nesting)
- **Header** – triggers grouping; when a grouping field changes value, the header prints; useful for group subtotals
- **Footer** – paired with a header; cannot exist without a header (though a header can be hidden while keeping its footer); used for subtotals per group
- Sections are added via the plus dropdown in the report structure panel on the left

Adding Fields to Extract Report Sections

- **Field types available:**
 - **Dataset field** – pulls a value from the report query output
 - **Expression** – calculated field using functions (eg date formatting, concatenation)
 - **Easy expression** – simplified interface for count, sum, average, max, min; used for totals in summary sections
 - **Constant** – static text typed directly; useful for institution name, contact info
 - **Variable field** – references a dashboard variable
- **Width** – every field has a width property (number of characters); for constants, type the value first, then set the width (the tool auto-counts characters; typing first prevents miscounting)
- **Pad alpha / pad numeric** – optional character padding for fixed-width fields when a system requires a specific fill character (eg dollar signs instead of spaces); only use when the spec requires it
- **Default extension** – set to **txt** for text file output
- **Byte order mark** – optional; check this box when output contains non-ASCII characters (eg accent marks, diacritics common in Middle Eastern or Irish names)

Title Section – State Upload

- Three fields: institution name (constant, 45 chars), contact email (constant, 45 chars), report date (expression)
- **Report date expression** – expression builder → FN (functions) → Date/Time category → **now()** function; returns the current system date and time
- **Key concept** – report-side functions (now, upper, etc) use a universal language regardless of database type; the data block SQL must match the database dialect (Oracle: **sysdate**; MS SQL: **getdate()**; PostgreSQL: **current_date**), but the report layer normalizes these differences
- **Date format mask** – applied via the formatting mask ellipsis; date/time section lists common formats; double-click to select and confirm it appears in the field below; format MMD-DYYYY or MM/DD/YYYY as specified; 10 characters including slashes

Detail Section – State Upload

- Fields: first name (25), last name (25), street line 1 (35), city (25), state (2), zip code (10)
- All dataset fields; drag the entire dataset header to add all fields at once, or double-click individually

Filters Tab

- Located at the report level (exit the section designer to access); allows additional filtering of records beyond what the dashboard parameters provide
- **Use case** – exclude records where state is blank, unknown (UNK), or a non-US code (eg KW for Kuwait, PU for a non-standard code)
- Filter condition: state is not null AND state not in ('UNK', 'KW', 'PU') – added via the

activate filtering checkbox and condition fields

Summary Section – State Upload

- One field: easy expression → count; width 8
- **Controlled by** – set to the detail section name; prevents the count from becoming disconnected if the data block is modified later (disconnected easy expressions display as zero with a strikethrough)

Crosstab Report

- **Use case** – display aggregate counts or sums without row-level detail; similar to a pivot table
- **Scenario** – count of employees per department
- **Report title** – added via the brackets icon → report title; creates a space at the top of the first page only (vs page header which appears on every page); place a text label inside with auto-width set to true to prevent truncation
- **Crosstab object** – added from the chart icon in the top toolbar; click in the canvas below the title area
- **Crosstab editor** – similar to a pivot table builder:
 - Bottom left quadrant: grouping field (eg department description)
 - Bottom right quadrant: value field (eg ID); change aggregate from Sum to Count
 - Top right quadrant: optional additional dimension (eg gender) for column breakdowns
- **Style** – select style offers preset color schemes (eg blue line); can be changed later
- Column headers in the crosstab can be renamed by double-clicking the header cell in the designer
- Preview via the magnifying glass icon or Control+F; adjust alignment (left, center, right) per column

Sample Data Blocks Folder

- Gail created sample data blocks available in each intern's folder for reference and project use:
 - **Argos Training Certificate generator** – banded report that takes trainer name, attendee names, course name, and date as dashboard inputs and produces a PDF certificate for each attendee; demonstrates using dashboard inputs rather than database data as the source
 - **Name tag generator** – similar input-driven approach; outputs formatted name tags based on an Avery label layout
 - **Student data (one row per student per term)** – sample student data block for use in intern projects
 - **Student data (one row per student per class)** – shows class-level enrollment detail per student
- Banded reports (covered in the next session) are required to understand the certificate and name tag data blocks; these will be reviewed on the final training day

Key Terms – Advanced Argos

- **Revision Control** – version history for a data block; enables reverting to a prior saved state; enabled per data block via security settings
- **Union Query** – combines the results of two or more SELECT statements; types: union (distinct), union all, intersect, minus
- **sys.dual** – a single-row, single-column Oracle system table used as a dummy table when a union query needs a table reference but has no real source data
- **Conditional Group** – a grouped set of where clause conditions in the visual design tool; OR

conditions must be wrapped in a conditional group to ensure correct SQL parenthesization

- **Checkbox** – dashboard object for boolean input; checked value and unchecked value properties define what is passed to SQL when each state is active
- **OnClick** – property on buttons, labels, and images that triggers an action (eg activate form) when the object is clicked
- **Visible** – property controlling object display; can be bound to a variable so the object only appears when that variable is populated
- **Enabled** – property that grays out an object until conditions are met; most often used on buttons
- **SQL Variable** – a named SQL snippet defined in the variables tab; can be used as a dataset for charts or referenced elsewhere in the data block without rebuilding the query
- **Standard Chart** – the web-friendly chart type introduced to replace the legacy classic chart; recommended for all new data blocks
- **Extract Report** – report type with three sub-formats (fixed-width, delimited, XML); uses sections (title, detail, summary, child, header, footer) to structure output; common for system-to-system data uploads
- **Easy Expression** – simplified expression interface in extract reports for aggregate functions (count, sum, average, max, min); used in summary or footer sections
- **Constant Field** – a static literal value added to a report section; used for institution name, contact info, or other non-data text
- **Formatting Mask** – pattern applied to date or numeric fields in extract reports to control output format (eg MM/DD/YYYY)
- **Crosstab Report** – report type that displays aggregated data in a grid without row-level detail; similar to a pivot table
- **now()** – universal report-layer function for the current date and time; works regardless of the underlying database type (unlike data block SQL, which must use the database-specific syntax)

6/18 – Argos Workshop Banded Reports & Wrap-Up

Argos Workshop 5 – Banded Reports

Banded Report Overview

- **Banded reports** – the most fully formatted Argos report type; typically output as PDF; support images, color, fonts, charts, and precise layout control
- Use cases: anything that must match an exact visual specification – contracts, award letters, invoices, certificates, schedules, regulatory documents
- Designed as a page layout, not a single flat file; the designer controls exactly what appears on the first page, every page, and the last page
- Band report bands are similar to extract report sections but with additional options; extract report has 6 section types, banded reports have 11

Band Types

- **Detail** – one row per record; must exist in every banded report (even if empty); the only required band
- **Page header** – prints at the top of every page; option to suppress on first page if a full title page is desired
- **Page footer** – prints at the bottom of every page; page numbers, data block/report name, print date; independent of page header

- **Column header** – similar to CSV column headers; auto-populated by the wizard when fields are selected; can be disabled once columns are moved elsewhere
- **Title** – prints once at the very top of the first page (below any page header); used for logos, branding, report titles
- **Summary** – prints once at the very bottom of the last page (above any page footer); used for grand totals
- **Group header** – prints when a grouping field value changes; must be paired with a group footer to enable subtotals; requires specifying the grouping field
- **Group footer** – prints after all detail rows for a group; used for subtotals; requires reset after print on count/sum expressions to prevent running totals
- **Child band** – highly flexible; used when other bands cannot provide required functionality; parent can only have one child, but children can have children (multi-level nesting)
- **Sub-detail band** – used when data from a second data source is needed, or for alternate formatting of existing data
- **Loop band** – repeats a band a specified number of times; use case: printing multiple copies of a label per record, or two copies of a W-2 (one for government, one for employee)

Banded Report Wizard

- Appears only when creating a brand new banded report; cannot be re-entered for an existing report
- **Six starting templates:**
 - **Add from library** – starts from a saved template; only available if templates exist in the library
 - **List report** – page header, title, column header, detail; best starting point for most list-style reports since the wizard auto-populates column headers
 - **Tabular** – page header, title, detail; no separate column header band (built into detail)
 - **Blank** – no bands; build everything from scratch
 - **Four-style** – one detail row per page; used for invoices, letters, or anything where each record gets its own page
 - **Avery labels** – wizard specifically for address labels or name tags based on standard label sheet layouts
- **List report** is the most common starting point since the wizard creates column headers automatically; blank report requires manually adding and labeling every field
- **Field selection step** – pick fields and set column header names; use the pencil (AB icon) to make labels editable; fields can be reordered with up/down arrows; all fields default to equal width across the page
- **Band selection step** – additional bands can be checked/unchecked; column header and detail are grayed out (cannot be removed via wizard); page footer is not checked by default and must be added manually here or after
- **Font formatting step** – per-band font defaults can be set here; detail band, column header band, title band each configurable; can also be done in the report designer later
- **Page settings step** – paper size, margins, orientation (portrait or landscape), columns per page (for multi-column directory-style output), report title text

Adding and Managing Bands After the Wizard

- **Create a new band** – blue rectangle icon at top of toolbar (hover: "create a new band"); opens a dialog to set the band type via dropdown (eg summary, page footer); band name defaults to "band N" and should be renamed

- **Special band icons** – three additional icons for bands with required configuration:
 - Child band icon – requires specifying the parent band
 - Group band icon – requires specifying the grouping field or expression
 - Sub-detail band icon – requires specifying the alternate data source
- **Renaming bands** – select the band, then click the gear icon (or right-click → Options); band names cannot contain spaces; use underscore or camelCase
- **Disabling a band** – right-click → Enabled toggle; disabled bands remain in the structure but produce no output; useful for keeping the column header band available for the wizard without printing it
- **Deleting a band** – select it, then use the circle-X button; confirm with OK
- **Editing a band** – double-click, right-click → Edit, pencil icon on the left panel, or double-click the band name in the left panel

Banded Report Field Types

- **System data field (SYS icon)** – pre-built expressions for the most common dynamic values: report date, report time, page number, detail count; easiest way to add dates and page numbers; format is fixed (cannot be customized)
- **Expression (E = MC² icon)** – full expression builder; supports functions, database fields, variables, and concatenation; allows date format masks, conditional logic, and multi-field combinations; use for anything more complex than what SYS provides
- **Data field (A with database icon)** – inserts a raw data field from the report query; used in detail and group bands to display record-level data
- **Label (A icon)** – static text; no single quotes required; for headers, captions, or fixed text
- **Image (sun/mountain icon)** – insert an image from file or library; set stretch mode to "keep aspect ratio" to avoid distortion; check auto-size report element so the full image displays; resize to approximately the desired size before importing

System Data Field – Common Types

- **Date** – current date in a fixed format; text property adds a label prefix (eg "Report Date:"); include a trailing space so the date does not run into the label
- **Time** – current time
- **Date/Time** – combined current date and time
- **Page number** – current page number; text property adds prefix (eg "Page "); for "Page X of Y" format use an expression instead
- **Detail count** – count of detail rows; used in summary band for grand totals; cannot be used in group footer (does not reset per group); for group-level counts use an expression with reset after print checked

Expressions in Banded Reports

- Expression builder accessed via dot-dot-dot on the expression field; three source types: database field, function, variable
- **Concatenation** – use + (not Oracle's ||) in the report layer; single quotes wrap literal text; example: 'Total employees for ' + title + ':' where title is a database field
- **Validate button** – checks expression syntax; variables will always throw an error during validation even when correct; do not panic on variable validation errors
- **Conditional expression** – example for optional instructor field: if instructor last name is empty, return blank; otherwise return last name + ", " + first name; prevents stray commas when data is absent

- **Reset after print** – checkbox on count and sum expressions in group footers; resets the counter/total for each new group; forgetting this causes running totals that accumulate across all groups
- **Get variable** – accesses system variables in expressions; `$datablock.name` returns the current data block name; `$report.name` returns the current report name; these update automatically if the data block or report is renamed

Alignment in Banded Reports

- Alignment toolbar works only within the same band; cannot align objects across different bands using the toolbar
- **Cross-band alignment** – use the gear/options to note the exact left position of a field in one band, then set the same left position on the corresponding field in another band
- **Blue guide lines** – appear while dragging; indicate alignment with objects in other bands
- **Auto-stretch height** – property on detail band fields; if a value exceeds the field width it wraps to the next line instead of being cut off; useful for long addresses or city names
- **Link with band** – band property that keeps the band paired with another band during pagination; prevents group headers from printing at the bottom of a page without their corresponding detail and footer
 - Header band: link with detail band
 - Detail band: link with footer band
 - Footer band: link with summary band
- **Re-print on new page** – group header property; if a group spans multiple pages, the header re-prints at the top of each continuation page so readers always know which group they are viewing
- **Force new page before** – group header property; forces each new group to start on its own page; used in per-person reports (schedules, letters) where each record should be self-contained

Page Footer – Recommended Content

- **Band frame** – add a top border line via the frame section when editing the band; visually separates the footer from the detail rows above
- **Page number** – SYS field, type: page number; placed right-aligned
- **Data block name** – expression using `$datablock.name` variable; label + expression pair for readability
- **Report name** – expression using `$report.name` variable; label + expression pair
- **Rationale** – if a colleague hands someone a printed report, the footer identifies exactly which data block and report produced it, enabling the recipient to regenerate or update it

Grouping in Banded Reports

- **Sort order must match grouping** – banded reports sort first, then group; if the sort order is not by the grouping field first, records from the same group will be scattered across the report
- **Group band creation** – use the blue-on-top/white-on-bottom icon; specify the grouping field via database field selection; creates a header band and an optional footer band
- **Group header** – displays the group name/value at the start of each group; use a data field to show the grouping value dynamically
- **Group footer** – displays subtotals or counts at the end of each group; use an expression with Count function and reset after print checked; add a top frame to visually separate it

from detail rows

- **Styling group headers** – can use background color and white font text for visual separation; copy column header labels into the group header band to provide repeated column context; disable the original column header band once labels are moved
- **Running total prevention** – in the group footer, the count/sum expression must have reset after print = yes; without this, values accumulate across all groups (eg group 1 shows 5, group 2 shows 14 instead of 9)

Address List for Employees – Banded Report Build

- **Bands used:** page header (report date), title (logo + data block name), column header (disabled after moving labels), detail (employee address fields), group header (department name, styled in pink with white text), group footer (count per department), summary (grand total), page footer (page number, data block name, report name)
- **Sort order:** department description first, then last name, then first name (grouping by department requires department to sort first)
- **Grand total:** SYS detail count in summary band with top frame
- **Subtotals:** expression Count in group footer with reset after print checked; label constructed as 'Total number of employees for ' + [dept field] + ':'
- **Field widths:** zip code and state narrowed; address and city widened; auto-stretch height enabled on address and city fields to handle outliers without truncation
- **Page break management:** link with band set on header-detail-footer chain; re-print on new page set on group header so multi-page groups always show the department name

Student Schedule – Per-Person Report Example

- Data block: student schedule; parameters: term, last name search (edit box), multi-select student list box; report query returns one row per class per student
- **Design decisions:**
 - Logo placed in page header band (not title band) so every student's page shows the logo, not just the first page
 - Grouping on student ID; force new page before set so each student gets their own page
 - Group footer sums credit hours (reset after print) and counts classes (reset after print)
 - Instructor field uses conditional expression: if instructor last name is null, display blank; otherwise display last name + ", " + first name; prevents stray commas for unassigned sections
 - Report header expression dynamically displays the selected term: 'Student Schedule for ' + [term description]
- **Scheduling integration** – Argos has a built-in scheduler; reports with email address data can be scheduled to run and distribute automatically; each student would receive only their own schedule

Argos Workshop Final – Library, Templates, and Wrap-Up

Library Objects in Banded Reports

- Dashboards and banded reports both use the library, but their saved objects are separate and not interchangeable; a logo saved from a dashboard cannot be retrieved into a banded report and vice versa
- **Saving to the library** – select the band or object, click the book-with-plus icon; choose a folder (personal or all users); provide a name and description
- **Retrieving from the library** – book-without-plus icon → navigate to the folder → select

the object; the object is placed on the current band

- **Common objects to save:** page footer band (data block name, report name, page number – generic enough to reuse across any report), logos, styled panels
- Saving generic objects to the library reduces setup time for new reports and enforces a consistent look and feel without manual repetition

Report Templates

- **Template** – a fully-structured banded report saved to the library with all bands, branding, and formatting in place but no data-specific content (fields, grouping fields removed); a starting point where the designer only needs to add data
- Templates appear with a T icon in the library; they are the only objects selectable via the "Add from library" option in the banded report wizard
- **Creating a template:**
 - Build a report with the desired band structure and branding
 - Make data-specific content generic: replace static text titles with `$datablock.name` variable expressions; remove grouping fields from group headers; remove data fields from detail and group bands; change specific text labels (eg "Total for [dept]") to generic versions (eg "Total count for group")
 - Re-enable the column header band (so the wizard can populate it for new reports)
 - Select the word "report" at the top of the structure panel on the left – nothing else
 - Click book-with-plus to save; the entry appears with a T icon
- **Portrait vs landscape templates** – save separate templates for each orientation; landscape requires reconfiguring element positions after changing orientation since Argos does not reflow content automatically
- Using a template means the designer focuses entirely on data – bands, branding, report date, footer, and styling are already done

Styles

- **Style** – a named color and font scheme saved in the library manager; applied to a banded report to set default fonts and colors per band type without manually configuring each band
- Created via the library manager (book icon on the main toolbar when fully exited from a report) → select a folder → click the Hollywood squares (grid) icon → name the style
- **Font settings** – configurable per band type: Normal (detail band), Heading 1 (title band), Heading 2 (group bands and summary band); Labels setting is not used in banded reports
- **Color settings** – four color slots: Color 1 (title + summary bands), Color 2 (group header/footer bands), Color 3 (detail band), Color 4 (not used in banded reports); both background color and font color are configurable per slot
- **Important:** the library manager does not remember custom RGB colors between sessions; re-enter RGB values each time; keep RGB codes noted separately
- Styles and templates can be used together or independently; a style replaces the per-band font/color setup step; a template replaces the band creation and layout step

Rebranding Data Blocks

- **Use case** – Evisions rebranded in 2015 and needed to update all client-facing data blocks; some had watermark images and compound backgrounds that made simple color changes insufficient
- **Process for rebranding a dashboard:**
 - Select all current parameter objects and labels (Control+click or shift-click)

- Save the group to the library (book-with-plus)
- Delete the existing objects from the dashboard
- Change the background panel color to the new brand color
- Retrieve the new branded template from the library (book-without-plus)
- Retrieve the saved parameter objects from the library and place them on the new template
- This approach avoids rebuilding SQL and variable connections from scratch; only the visual layer changes

Training Environment Access

- Training environment (training.evisions.com) will remain available through the end of August
- Interns have their own folders and can continue building data blocks and reports for capstone projects or independent practice
- Sample data blocks folder placed in each intern's folder includes: student schedule, student data (one row per student per term), student data (one row per student per class), financial aid award letter, training certificate generator, name tag generator
- Argos X and the dashboard designer are accessible in the training environment; banded reports, CSV, extract, and crosstab reports can all be built and tested
- Argos scheduler is available in the training environment; reports can be scheduled to run and email output to specified recipients

Key Terms – Banded Reports and Library

- **Banded Report** – the most fully formatted Argos report type; PDF output; supports all band types, images, color, and conditional logic
- **Band** – a container row in a banded report that controls when and where its content prints (eg every page, first page only, per group)
- **Title Band** – prints once at the top of the first page; used for logos and report titles
- **Summary Band** – prints once at the bottom of the last page; used for grand totals
- **Group Header / Footer** – bands that bracket a set of detail rows sharing a common grouping field value; header prints when the group starts, footer prints when it ends
- **Child Band** – a flexible band that follows its parent; used when other band types cannot provide the needed layout behavior
- **Loop Band** – repeats its content N times per parent row; used for duplicate copies of a document
- **Force New Page Before** – group header property that starts each group on a new page; used for per-person documents
- **Re-print on New Page** – group header property that repeats the header at the top of each continuation page when a group spans multiple pages
- **Link With Band** – band property that keeps logically related bands together during pagination; prevents footer-only-on-next-page breaks
- **Reset After Print** – property on count/sum expressions in group footers; resets the counter for each new group to prevent running totals
- **SYS Field** – system data field shortcut for common expressions (date, time, page number, detail count); format is fixed
- **Auto-Stretch Height** – field property that allows a field to expand vertically if its value exceeds the field width; prevents truncation of long values
- **Template** – a banded report structure saved to the library with branding and band layout but no data-specific content; T icon in the library; selectable via the "Add from library"

wizard option

- **Style** – a named color and font scheme stored in the library manager; applied to a banded report to set default appearance per band type
- **\$datablock.name / \$report.name** – system variables that return the current data block or report name dynamically; used in expressions for automatic labeling